



Pimpri Chinchwad Education Trust's
**Pimpri Chinchwad College of Engineering
(PCCoE)**
(An Autonomous Institute)
Affiliated to Savitribai Phule Pune
University(SPPU) ISO 21001:2018 Certified by
TUV



Course: Software Testing and Quality Assurance Laboratory	Code: BIT26PE11
Name: Amar Vaijinath Chavan	PRN: 124B2F001
Assignment 1: Write a black box functional test cases using manual testing for the given application	

1. THEORETICAL BACKGROUND

1.1 Black Box Testing Approach

Black Box testing is a software testing methodology where the internal code structure, implementation details, and internal paths are not visible to the tester. The test process is driven entirely by software requirements and functional specifications.

- **Focus:** It focuses strictly on the inputs provided to the system and the resulting outputs.
- **Input Selection:** Since testing every possible input is impossible, testers use strategic techniques to select a representative subset of data.
- **Random Testing:** Selecting test inputs purely by chance from the input domain is known as random testing. For example, if a room price field accepts values between 1 and 10,000, a random test might use 55, 24, or 3.

1.2 Black Box Testing Techniques

To maximize defect detection with a limited number of test cases, the following techniques are applied:

A. Equivalence Class Partitioning (ECP)

- This technique divides the input domain into classes of data where a test value in a particular class is equivalent to any other member of that class.
- **Example:** In a Hotel Management system, if a "Number of Guests" field accepts 1 to 10 people, any value in this range (e.g., 5) represents the "Valid" class. Values like 0 or 11 represent "Invalid" classes.

B. Boundary Value Analysis (BVA)

- BVA refines ECP by focusing on the "edges" of input ranges where errors often occur.
- **Key Values:** LB (Lower Boundary), UB (Upper Boundary), and values immediately adjacent (ALB, BLB, AUB, BUB).
- **Example:** For a Room ID length requirement of 3 to 15 characters, BVA tests lengths of 2 (BLB), 3 (LB), 4 (ALB), 14 (BUB), 15 (UB), and 16 (AUB).

C. Cause-Effect Graphing

- This technique illustrates the logical relationship between specific input conditions (**Causes**) and the resulting actions (**Effects**).

- **Logic:** The specification is transformed into a graph resembling a digital logic circuit to disclose inconsistencies.

D. State Transition Testing

- This focuses on the internal configurations (states) of a component and the events that trigger changes between those states.
- **Example:** A "Room" status might change from *Available* to *Booked* upon a successful transaction, or from *Booked* to *Cancelled*.

2. COMPREHENSIVE TESTING CATEGORIES

2.1 GUI (Graphical User Interface) Testing

GUI testing ensures that the visual elements of the Hotel Management system are functional and aesthetically correct.

- **Layout:** Verifying that the navigation bar (Home, Rooms, Facilities) is aligned and functional.
- **Visuals:** Checking button hover effects, color consistency (e.g., Teal theme), and image resolutions in the room carousel.
- **Responsiveness:** Ensuring the Admin Dashboard and User Modals (Login/Register) adjust correctly to different screen sizes.

2.2 Performance Testing (Load and Stress)

Since the project is assumed to be hosted on an **AWS EC2 t2.micro** instance (1 vCPU, 1 GiB RAM), performance testing evaluates system stability.

- **Load Testing:** Simulating a "Normal" expected number of users (e.g., 50 users) to ensure the page load time remains under 2.5 seconds.
- **Stress Testing:** Pushing the system beyond its limits (e.g., 200 concurrent "Book Now" clicks) to identify the server's breaking point and verify its recovery capability.

2.3 Error Guessing

This is an ad-hoc approach where the tester uses experience to "guess" where errors might occur.

- **Example:** Attempting to book a room with a "Check-out" date that is earlier than the "Check-in" date, or entering non-numeric characters in the phone number field.

3. CONCLUSION

In this assignment, **Black Box Testing** techniques were applied to the Hotel Room Booking and Management System. By using **Equivalence Class Partitioning** and **Boundary Value Analysis**, the testing process was optimized to detect critical defects with a minimal set of data. Detailed GUI and Performance (Load/Stress) theory ensured that the system is not only functional but also visually professional and ready for cloud deployment on environments like AWS EC2.

PROJECT TEST CASE DRIVE LINK:

https://drive.google.com/drive/folders/1p_eqXt37ShyiS9Ne9_XOHAvh2heI1EGI?usp=sharing